



harness engineering all you need

ハーネスエンジニアリングとは？

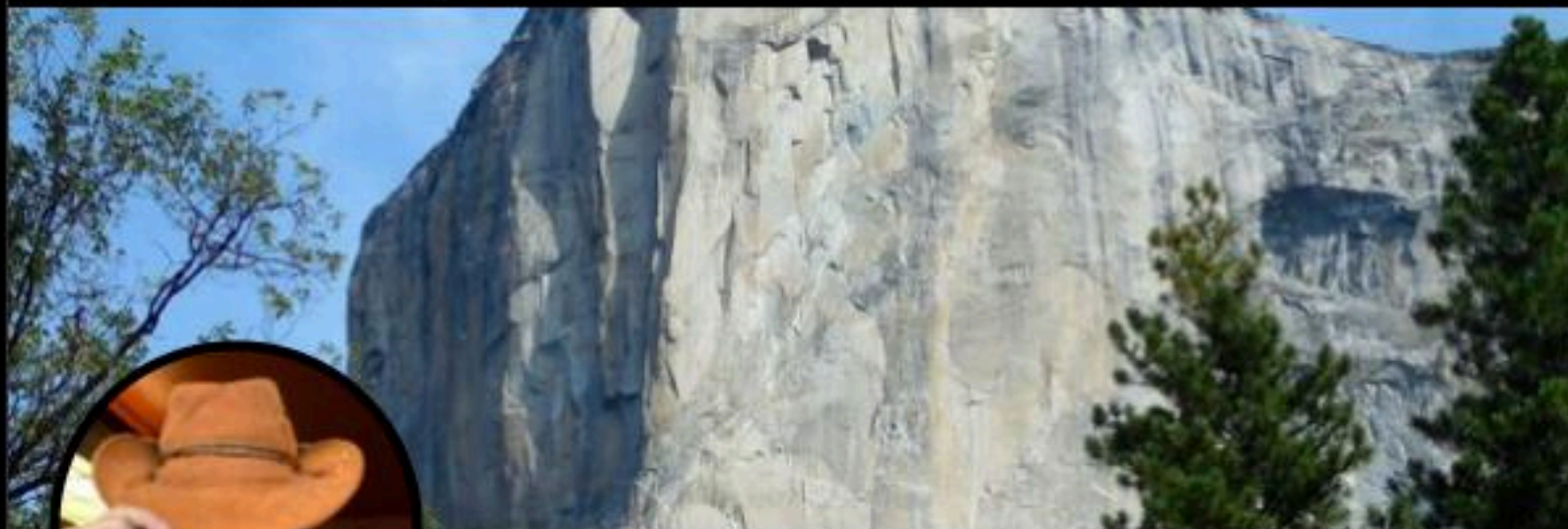
2026/4/24（金）

KINOPEE



Kinopee / きのぴー

18.2万 件のポスト



プロフィールを編集

Kinopee / きのぴー

@kinopee_ai

Cursor Ambassador / Devin Expert / Windsurf Ambassador / Sponsored by
CodeRabbit / 「Cursor完全ガイド」 著者。「Software Design」 誌で「プログラミングxAI の最前線」連載中。

HARNESSENGINEERING・2026

ハーネスエンジニアリング 5つの流派

実装が先、命名は後——定義の揺れを現場感覚で読み解く

5つの流派

同じ「ハーネス」を語っていても、論者が原器にしている実装で重心が違う

1

Chase派

広義・分類論

代表者

Harrison Chase
(LangChain)

中核主張

モデル以外の
すべてがハーネス

2

Hashimoto派

運用哲学

代表者

Mitchell
Hashimoto

中核主張

失敗したら環境を
再設計せよ

3

Fowler派

制御理論

代表者

Martin Fowler
(Thoughtworks)

中核主張

コンテキスト設計の
特殊形として定義

4

Chawla派

同心円モデル

代表者

Avi Chawla
(教育系)

中核主張

LLMのOS層
三重の入れ子

5

Codex派

スケール駆動

代表者

Ryan Lopopolo
(OpenAI Codex)

中核主張

スケールで壊れる
問題の答え

1

CHASE SCHOOL ・ BROAD DEFINITION

Chase派

広義ハーネス / 分類論

「モデル以外のすべて」を一括でハーネスとして囲い込む

代表者

Harrison Chase (LangChain創業者)

時期・出典

2025年10月 ブログ「Agent Frameworks, Runtimes, and Harnesses」

原器となった実装

DeepAgents (LangChain)

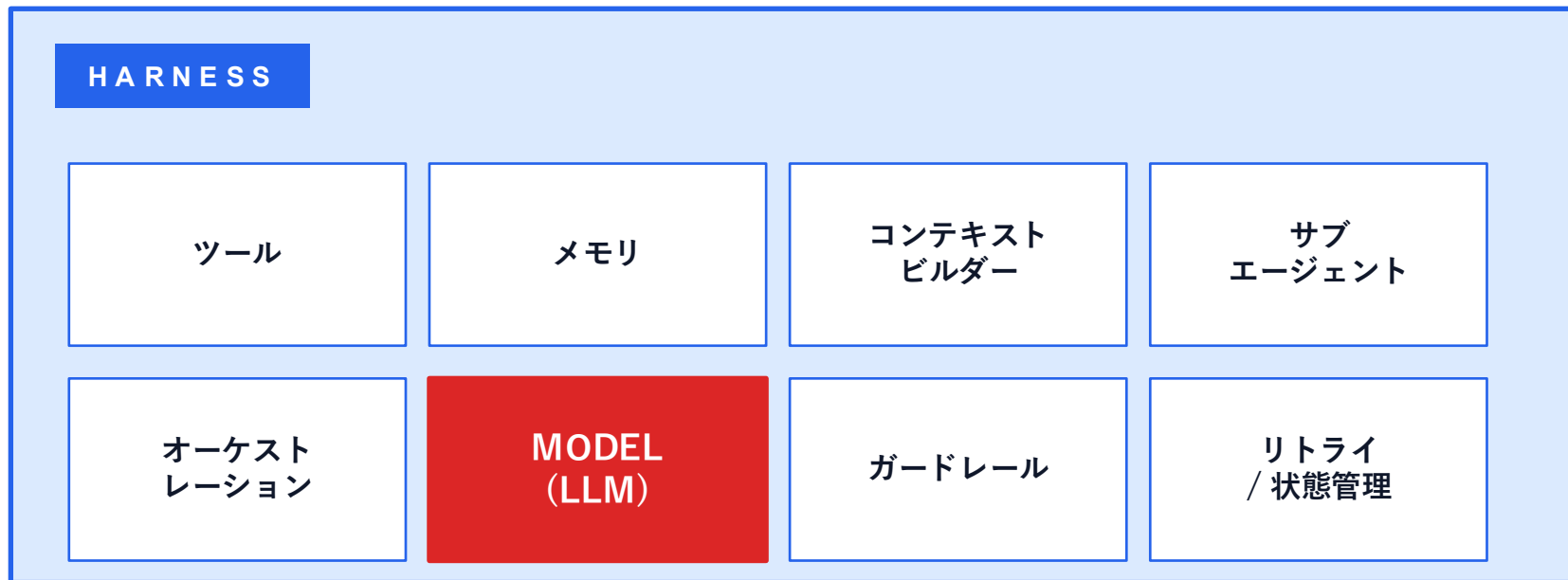
中核主張

- framework / runtime / harness の三分類を提示
- ツール実行・メモリ・サブエージェント・ガードレール・オーケストレーションまで全部ハーネスに畳み込む
- ハーネスは「コンテキストを生成する外側の層」であり、コンテキスト設計はその一構成要素
- 原器がDeepAgentsなので、総合フレームワーク設計の話として語られがち

特徴的フレーズ

***If you're not the model,
you're the harness.***

—— Vivek Trivedy (LangChain)



「If you're not the model, you're the harness.」—— モデル以外のすべての構成要素をハーネスとして囲い込む

2

HASHIMOTO SCHOOL ・ OPERATIONAL

Hashimoto派

運用哲学 / 失敗駆動

エージェントの失敗を、再発不能にする環境設計の累積

代表者

Mitchell Hashimoto (HashiCorp創業者)

時期・出典

2026年2月 自身のブログ記事 / OpenAI Codex事例と合わせて急速に普及

原器となった実装

AGENTS.md / CLAUDE.md / .cursorrules

中核主張

- エージェントが失敗したら、物理的に同じミスができないよう環境を再設計せよ
- AGENTS.mdに一行ずつ失敗防止ルールを追加していく「運用の累積」がハーネスエンジニアリング
- 設計物としてではなく、育てていくプロセスとしてのハーネスを語る
- 日本の.cursorrules運用者と地続きで、現場感覚に最も近い流派

特徴的フレーズ

***Agents aren't hard;
the Harness is hard.***

—— Ryan Lopopolo (OpenAI Codex, 同流派で引用)



ひ 新しいタスクで再開 —— 失敗知識が AGENTS.md に累積していく

失敗のたびに環境が強くなる —— ハーネスは「設計物」ではなく「育てていくもの」

3

FOWLER SCHOOL ・ CONTROL THEORY

Fowler派

狭義ハーネス / 制御理論

ハーネスはコンテキストエンジニアリングの特殊形 —— 包含関係を逆転

代表者

Martin Fowler (Thoughtworks,
Distinguished Engineer)

時期・出典

2026年 Thoughtworks記事「Harness
engineering for coding agent users」

原器となった実装

コーディングエージェント全般 (Claude
Code / Cursor / Devin)

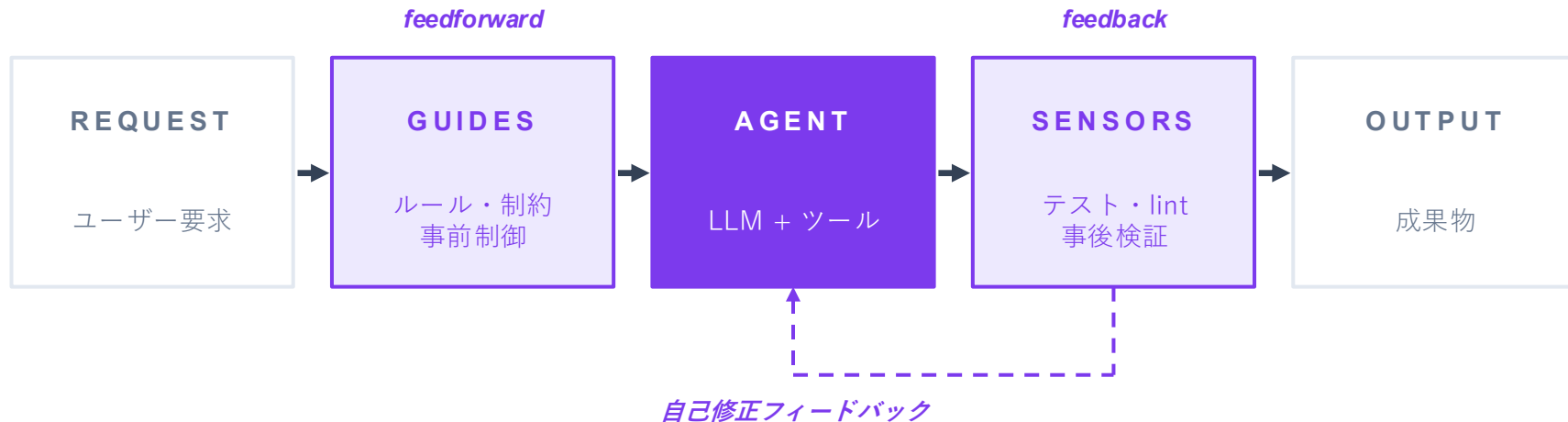
中核主張

- 他の流派と異なり、ハーネスをコンテキストの内側(特殊形)に位置づける唯一の派
- guides (feedforward制御) と sensors (feedback制御) の両輪でエージェントを信頼に足るものにする
- 「不要な出力を事前に防ぐ仕組み」と「逸脱を検知して自己修正させる仕組み」
- 制御工学の語彙 (フィードバックループ・センサー) に接続する理論志向

特徴的フレーズ

**ハーネス設計は
コンテキスト設計の
特殊な一形態である。**

—— Martin Fowler (趣旨)



guides (事前制御) × *sensors* (事後検証) の両輪でエージェントを信頼できる存在にする

4

CHAWLA SCHOOL ・ CONCENTRIC MODEL

Chawla派

同心円モデル / OS比喻

プロンプト ⊂ コンテキスト ⊂ ハーネスの三重入れ子で整理

代表者

Avi Chawla (Daily Dose of DS) / 教育系
ライター群

時期・出典

2026年前半「The Anatomy of an Agent
Harness」ほか

原器となった実装

特定ツールではなく概念モデル

中核主張

- プロンプト・コンテキスト・ハーネスを三重の同心円で整理する
- Beren Millidge (2023) のOS比喻を継承——LLMはCPU、コンテキストはRAM、ツールはデバドラ、ハーネスはOS
- 定義の厳密さより、初学者への説明しやすさを優先する立場
- Chase派と論理的には整合的だが、より図解寄り・入門記事向き

特徴的フレーズ

**LLMは
OSなしの裸のCPUだ。
ハーネスがOSに当たる。**

—— Beren Millidge (2023, 継承されるアナロジー)

4

CONCEPT DIAGRAM

Chawla派

同心円モデル / OS比喻

コンピュータ

OS

デバイスドライバ

ディスク

RAM

CPU



LLMエージェント

ハーネス

ツール

外部DB / メモリ

コンテキスト

LLM

"LLMはOSを持たない裸のCPUに過ぎない。ハーネスがOSの役割を担う"

5

CODEx SCHOOL ・ SCALE-DRIVEN

Codex派

スケール駆動 / プロダクション答え

プロダクションスケールで壊れる問題への答えとしてハーネスを定義

代表者

Ryan Lopopolo / OpenAI Codexチーム

時期・出典

2026年2月 OpenAIの事例公表 (3人で百万行、95%AI生成)

原器となった実装

OpenAI Codex / その内部ハーネス

中核主張

- プロンプトでもコンテキストでも解けない、多段ステップ・自律実行・並列特有の故障モードが存在する
- その故障モードを解くのがハーネス、というスケール駆動の動機づけ
- 他流派と直接対立はせず、ハーネス概念が必要になった理由を提供する立場
- 登壇の導入 (なぜ今この概念が要請されたか) に使いやすい

特徴的フレーズ

***Agents aren't hard;
the Harness is hard.***

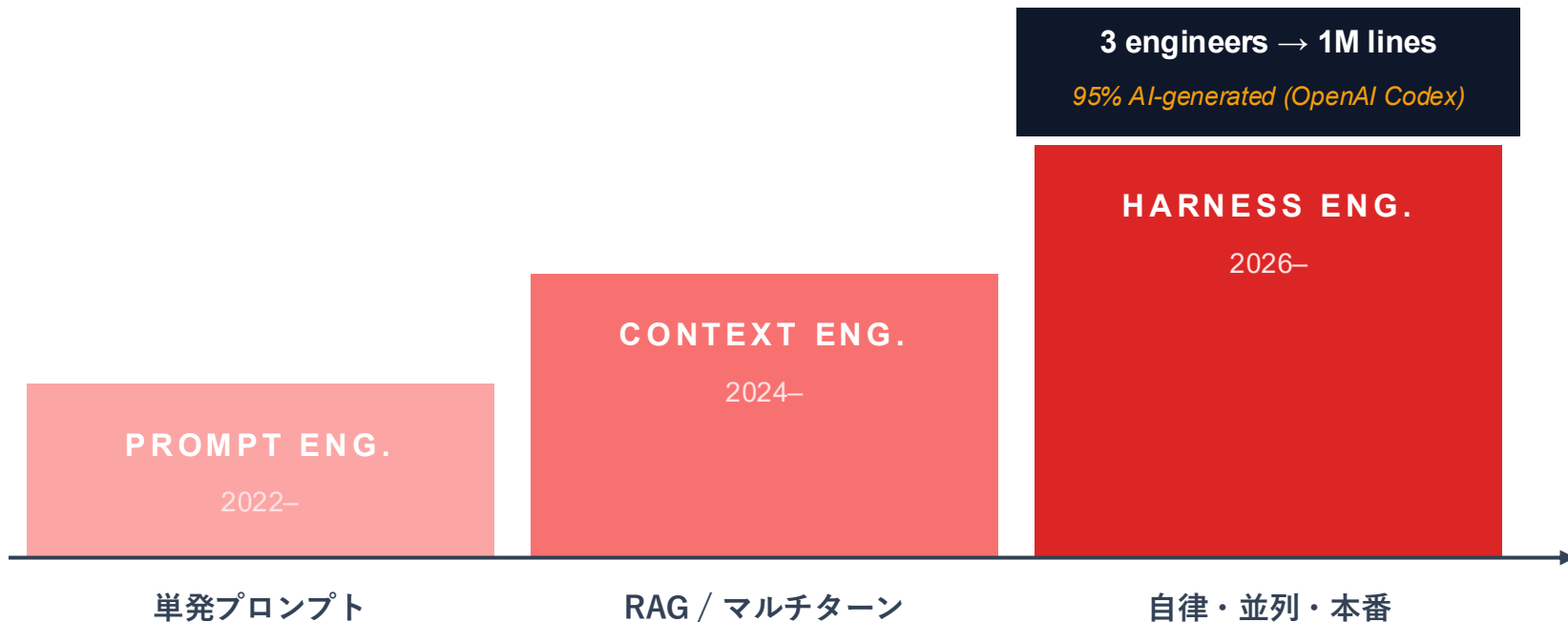
—— Ryan Lopopolo (OpenAI Codex)

5

CONCEPT DIAGRAM

Codex派

スケール駆動 / プロダクション答え



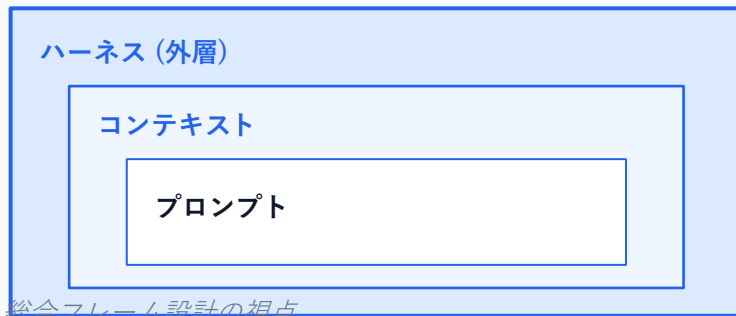
"Agents aren't hard; the Harness is hard." — プロダクションスケール特有の故障モードを解くのがハーネス

対立の構造

包含関係が逆転する —— 流派間のいちばん劇的な噛み合わなさ

Chase派 / Chawla派

ハーネス ⊃ コンテキスト



総合フレーム設計の視点

Fowler派

コンテキスト ⊃ ハーネス



制御理論 (feedforward / feedback) の視点

日本に紹介する際は、どちらの意味で語られているかを毎回明示する必要がある

ハーネスとガードレール

AIコーディングを支える、役割の違う2つの仕組み

HARNESS

ハーネス

= 馬具

- カスタムインストラクション / プラン
- Skills
- サブエージェント

GUARDRAILS

ガードレール

= 道を外れないための柵

- 静的解析 (lint, 型チェック)
- ビルド
- テスト

進路を示すハーネス、道を外れないよう守るガードレール —— これがAIコーディングの土台

ハーネス

= 馬具 —— 走り出す前に「どう走ってほしいか」を伝える層



馬具 (bridle & reins)

役割

走り出す前に「どう走ってほしいか」を馬具で伝える —— 事前設計の層

具体的な道具

- カスタムインストラクション / プラン
- Skills
- サブエージェント

私の主張

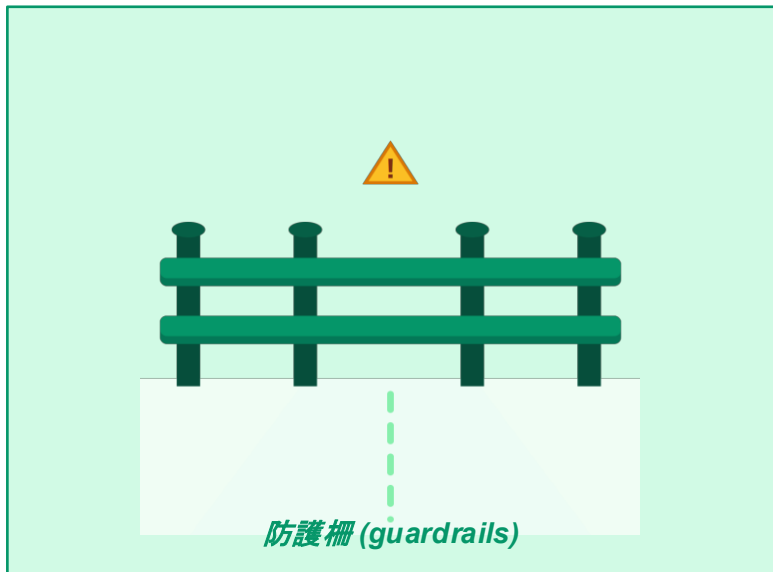
日本の現場は、名前がつく前から既にやっていた。

実装が先、命名は後 —— 現場感覚の延長線上に「ハーネス」という言葉が乗っただけ。



ガードレール

= 道を外れないための柵 —— 走った後に「逸脱していないか」を検証する層



役割

走った後に「道から外れていないか」を機械で自動検証する —— 事後検証の層

具体的な仕組み

- 静的解析 (lint, 型チェック)
- ビルド
- テスト

私の主張

ハーネスは主観、ガードレールは客観。

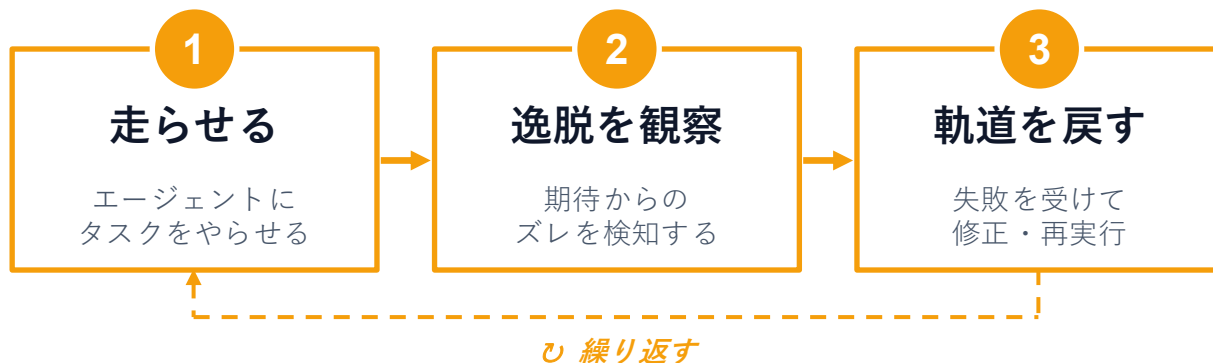
機械的な判定が強いほど、エージェントにハンドルを預けられる —— 自律性を上げる鍵はここ。



FEEDBACK LOOP

フィードバック・ループ

馬具と柵だけでは、馬は目的地にたどり着けない



具体的には

テストが落ちたら → 直させる

lint が出たら → 修正させる

型が通らなければ → やり直させる

—— だから、定義に拘るよりも、馬具も柵もループも、今使える手段を全部使って結果を出しに行く

CONCLUSION

言葉の定義は、重要ではない。

大切なのは、期待した結果を得ること。

今使える手段を総動員、

適切な場所で、適切に使って、良い結果を得ましょう。